Project - High Level Design
on
Chat with Technology Documents (RAG)

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Dishant Patil	EN22CS301338
2.	Devansh Solanki	EN22CS301324
3.	Bhumi Geete	EN22CS301273
4.	Bhavika Chandrawat	EN22CS301266
5.	Garv Patidar	EN22CS301360

Group Name: 05D3

Project Number: GAI-29

Industry Mentor Name: Mr. Suraj Nayak

University Mentor Name:: Prof. Vineeta Rathore

Academic Year: 2025-2026

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

9. Output & ScreenShots

1.Introduction

The Technology Documents RAG System is a domain-specific Retrieval-Augmented Generation (RAG) application designed to enable intelligent question-answering over energy-related PDF documents. The system allows users to upload technical energy documents such as compliance reports, operational manuals, sustainability reports, and research papers and interact with them through a conversational interface.

Unlike traditional keyword-based search engines, this system leverages semantic vector embeddings and Large Language Models (LLMs) to retrieve contextually relevant information. By combining FAISS-based vector similarity search with Ollama-powered text generation, the system ensures that responses are grounded strictly in the uploaded document content.

The application is implemented using Python and React for the frontend interface, FAISS for vector indexing, and Ollama APIs for embeddings and response generation.

Scope of the Document

This document describes the high-level architecture and technical design of the Technology Documents RAG System.

Included in Scope:

End-to-end RAG pipeline architecture

PDF ingestion and text extraction

Technology-domain document validation

Text chunking strategy

Embedding generation using Ollama APIs

FAISS vector database implementation

Out of Scope:

Development of proprietary LLM models

General-purpose document management outside the Technology domain

Mobile applications

Real-time collaborative editing

The focus is strictly on the design and functioning of an Technology-domain RAG system.

Intended Audience

This documentation is intended for:

Primary Audience:

Software Architects & Developers – Understanding system design and integration.

Data/ML Engineers – Implementing embeddings, chunking, and vector indexing.

Secondary Audience:

Technology Industry Professionals – To validate domain-specific outputs.

Academic Researchers – Studying RAG architecture implementation.

AI/ML Students – Understanding applied generative AI systems.

System Integrators – Evaluating integration with enterprise platforms

Prerequisites:

Basic knowledge of Python, APIs, NLP concepts, and vector search.

Familiarity with technology-sector documentation (for domain users).

System Overview

The system follows a modular RAG pipeline that transforms static Technology documents into an interactive knowledge base.

Core Workflow

Upload – User uploads energy-related PDF documents.

Parse – Text is extracted while preserving structure and metadata.

Chunk – Content is segmented into manageable, overlapping sections.

Embed – Each chunk is converted into a semantic vector representation.

Index – Embeddings are stored in a high-performance Vector Database.

Query – User submits a natural language question.

Retrieve – Top relevant chunks are selected using similarity search.

Generate – LLM produces a grounded answer using retrieved context.

Cite – Response includes references to source sections.

2.System Design

System Design – RAG AI Application:

- Application: Retrieval Augmented Generation (RAG) AI Chat App
- Purpose: Use vector search + LLM to answer user queries using uploaded docs
- Users: End-User, Admin
- Inputs: User Q, Document
- Output: Context-aware Answer

Application Design

Frontend – React UI for interaction

- Input box, Upload docs, Chat window

Backend – Python server

- Handles queries, document storage, search

Vector DB – Stores embeddings

LLM Service – Generates responses

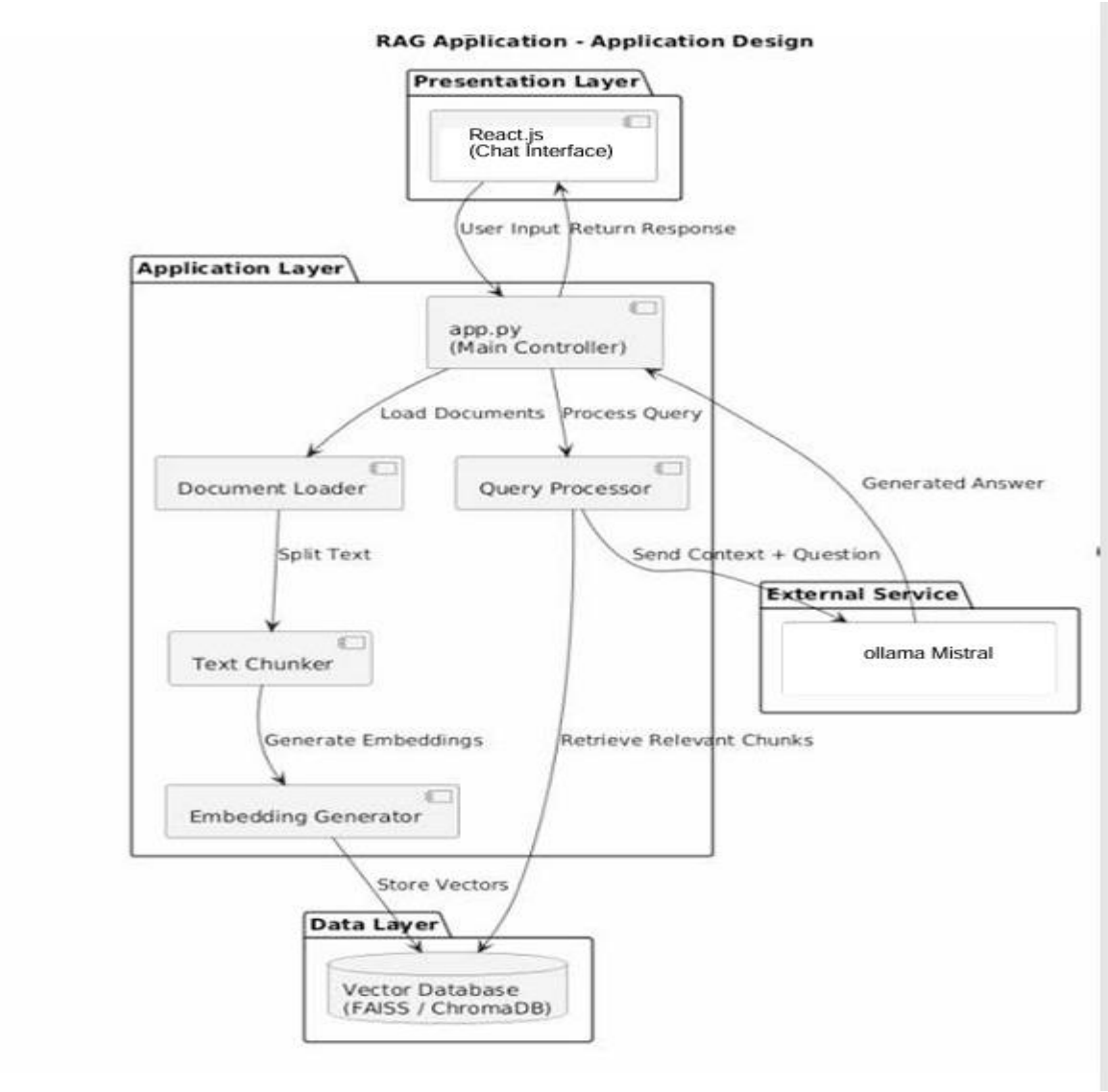


Fig:2.1-Application Design

RAG Process Flow:

Process Flow Explanation

The user opens the application and uploads one or more documents, which are converted into text and divided into chunks. These chunks are encoded into vector embeddings and stored in a vector database. When the user enters a query, a semantic search retrieves the most relevant document chunks, and the LLM generates an answer using this retrieved context.

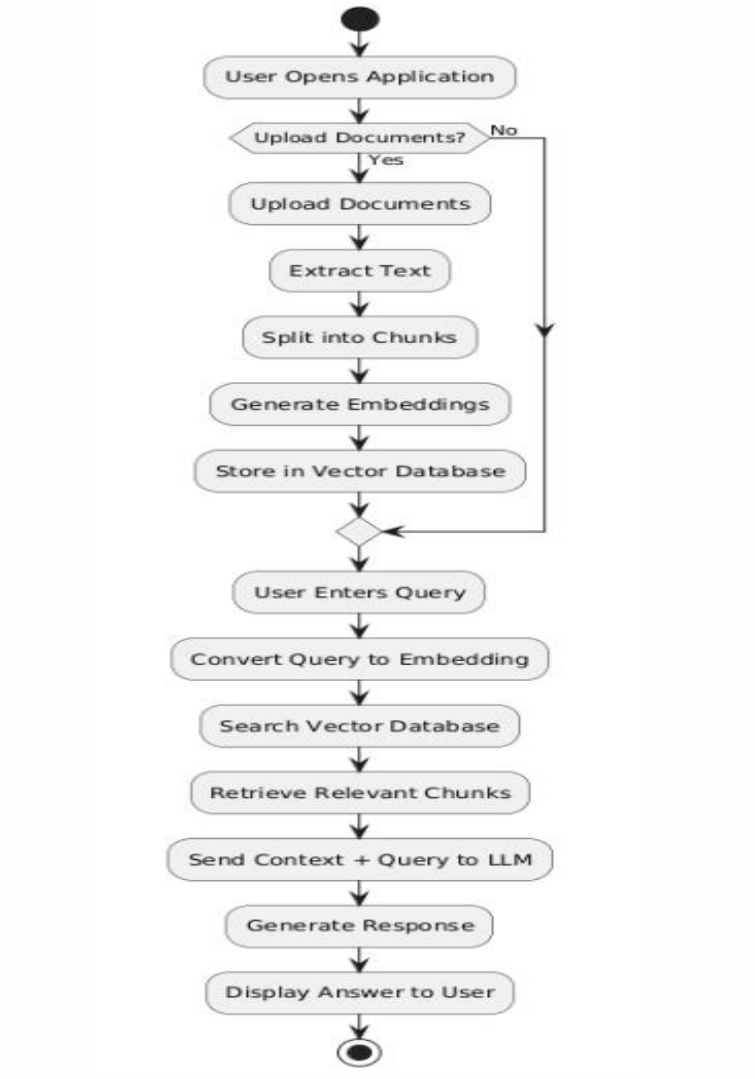


Fig:2.2-Process Flow

Information Flow

The system first takes the document content and user question as input. The document text is chunked and converted into vector embeddings, which are stored in a vector database. When a user asks a question, the system performs a vector search to retrieve the most relevant document chunks. These retrieved chunks are then passed to the LLM, which generates a context-based chat response as the final output

- **Data In**: Document content + User question
- **Processing**: Chunking → Embeddings → Search
- **Store**: Vector Database
- **Query**: Vector Search + LLM Answer
- **Output**: Chat response

Components Design

Component	Responsible
React UI	UI + chat interface
app.py	Request handler
Embeddings Module	Converts text to embeddings
Vector Store	Holds semantic search data
LLM Client	API calls for responses

Key Design Considerations :

- Scalability: handle large corpora
- Accurate Retrieval: semantic search quality
- Embedding choice & vector DB optimization
- User privacy & API keys

- UI responsiveness
- Caching repeated queries

API Catalogue:

If it's a REST backend (or pseudo-API style):

Method	Endpoint	Description
GET	/	Homepage
POST	/upload	Upload docs
POST	/ask	User question

3. Data Design

Data Model

This diagram shows how data moves inside a simple RAG(Retrieval-Augmented Generation) system.

- Document: The uploaded file that contains full text data.
- Chunk: The document is split into smaller parts to make searching easier.
- Embedding: Each chunk is converted into numerical vector format so the system can understand and compare it.
- UserQuery: The question asked by the user.
- Answer: The final response generated by the system based on the most relevant chunks.

Flow:

Document → Split into Chunks → Convert to Embeddings
User asks Query → System searches relevant Chunks → Generates Answer

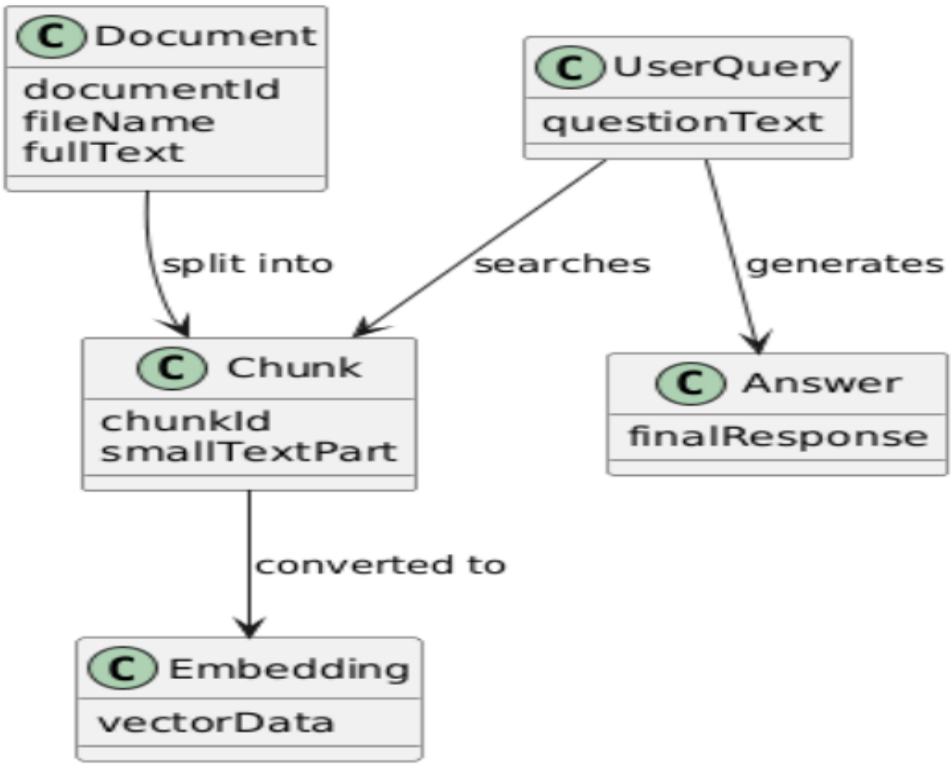


Fig:3.1-Data Model

4. Interfaces

The system provides a **React user interface** that allows users to upload Technology-related documents, submit queries, and view generated answers. It interacts with external services through **HuggingFace Embeddings and Chat Completion APIs** for semantic processing and answer generation. Internally, the system integrates a **FAISS vector index** to enable fast and accurate semantic document retrieval, along with local fallback mechanisms to ensure uninterrupted operation.

---

5. State and Session Management

The application follows a **stateless execution model**, where all data such as document text, embeddings, and query results exist only during runtime. Each user request is processed independently, ensuring simplicity and scalability, and no session data is persisted once the execution cycle completes.

---

6. Caching

The system does not employ explicit caching mechanisms for long-term storage. However, during runtime, the **FAISS vector index and retrieved document chunks** remain in memory and are reused within the same query execution, providing temporary performance optimization without compromising data freshness.

---

7. Non-Functional Requirements

7.1 Security Aspects

Security is ensured by managing all API credentials through **environment variables**, preventing hardcoded secrets in the source code. Additionally, the

system restricts the language model to generate responses **strictly from the retrieved document context**, thereby minimizing the risk of hallucinated or unauthorized information.

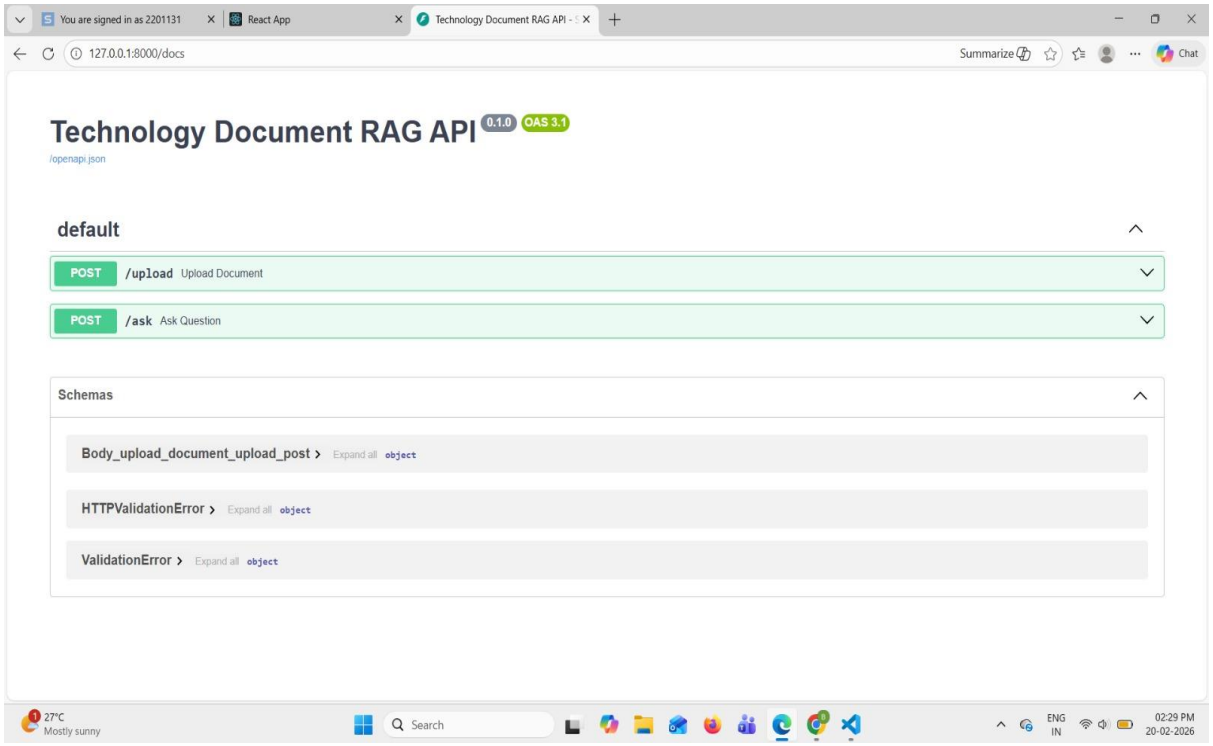
7.2 Performance Aspects

The system achieves efficient performance through the use of **FAISS for high-speed vector similarity search**, enabling rapid retrieval of relevant document content. Overall response time depends on document size and external LLM API latency. To maintain reliability, the system incorporates **fallback strategies**, such as local embeddings and extractive answer generation, in cases of API rate limits or failures.

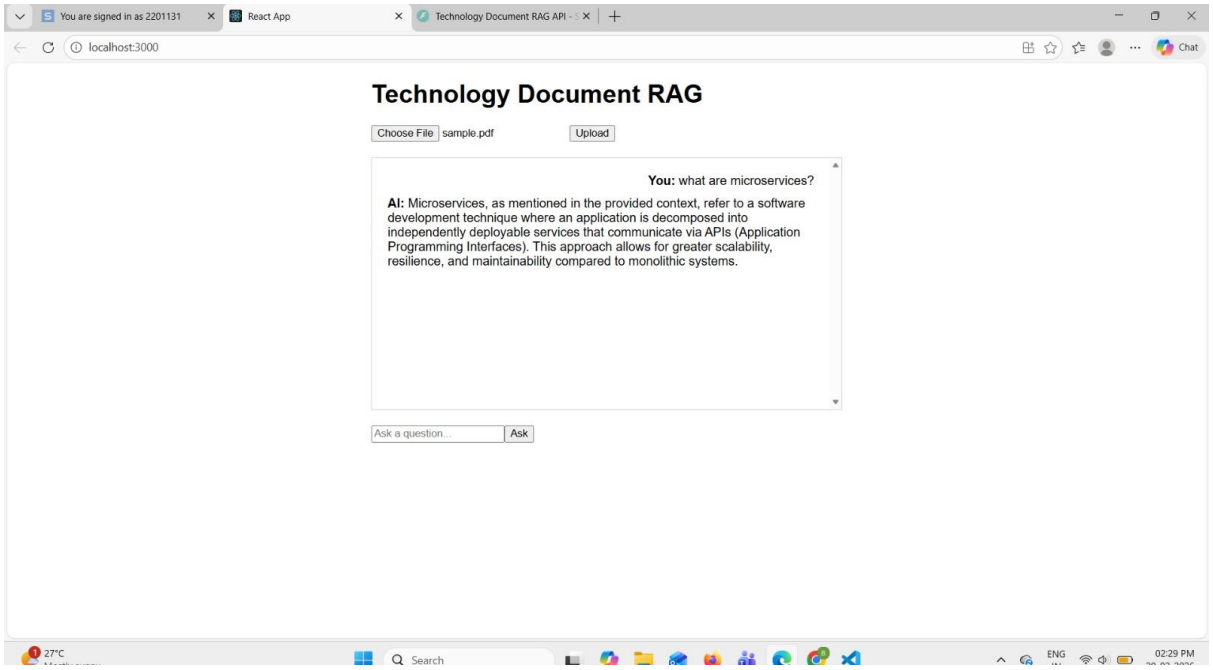
8. References

- React Documentation
- Ollama API Documentation
- FAISS Library Documentation
- PyPDF Documentation
- Retrieval-Augmented Generation (RAG) Research Papers

9 Output & ScreenShots :-



4.1 Backend



4.2 Frontend

Tab 3